

NEHA SHARMA - U24AI050 LAB-5

16 February 2026 14:46

1. Write a Simulator which runs different CPU scheduling algorithms and gives different Utilization matrices viz.

- a. CPU Utilization
- b. Waiting time of each process and average waiting time
- c. Response time of each process and average response time
- d. Turn-around time of each process and average turn-around time

Your simulator will take only two arguments – 1) Scheduling Algorithm 2) No. of Processes. On the basis of these two arguments you need to randomly generate multiple internal parameters. Some of the parameters are

- 1) Arrival Time
- 2) CPU Burst or Execution time
- 3) Deadline

Scheduling algorithms which you need to simulate are

- 1) FCFS
- 2) RR
- 3) SJF
- 4) SRTN
- 5) Priority or additionally any other algorithm if you want to include.

User may want to see results for all the algorithms in the single run. In that case, you need to use the same data for all the algorithms. Some of the points to ponder

- 1) Use a random number generator which is near a perfect random number generator.
- 2) In SJF there is no need of Arrival time. Consider all the jobs' arrival at the start.
- 3) In RR, you may show your results for multiple quantum sizes.
- 4) Preemptive and Non-Preemptive results need to be shown for different algorithms.
- 5) Your example run will look like this.

SVNIT, DoAI Operating System Lab

Result

```
root# CPUSimulator
```

Hi! Welcome to CPU Scheduling Simulator. Please give me required parameters.

Scheduling algorithm – (1) FCFS (2) RR (3) SJF (4) SRTN (5) Priority (6) All
ALL

No. of Processes in the system : 50

Wait.... Generating Schedules...

DONE. Please check output file Output. tex for all the results.

You can produce whole output at STDOUT also but due to a large number of processes you need to use a file for output. You may also use different files for different purposes. You need to create a file process <studentid>_data.tex which contains randomly generated data for given no. of processes.

Additional features.

1) Gantt charts

2) Comparison table at the end for all the algorithms.

```
-----  
#include <bits/stdc++.h>  
using namespace std;  
  
struct Process {  
    int pid;  
    int arrival;  
    int burst;  
    int remaining;  
    int completion;  
    int waiting;  
    int turnaround;  
    int response;  
    int priority;  
    int deadline;  
    bool started = false;  
};  
  
mt19937 rng(random_device{}());  
  
int randomInRange(int low, int high) {  
    uniform_int_distribution<int> dist(low, high);  
    return dist(rng);  
}  
  
vector<Process> generateProcesses(int n) {
```

```

vector<Process> p(n);
for(int i=0;i<n;i++) {
    p[i].pid = i+1;
    p[i].arrival = randomInRange(0,10);
    p[i].burst = randomInRange(1,10);
    p[i].remaining = p[i].burst;
    p[i].priority = randomInRange(1,5);
    p[i].deadline = randomInRange(10,30);
}
return p;
}

void printMetrics(vector<Process> &p, string name, ofstream &out) {
    double total_wait=0, total_turn=0, total_resp=0;
    int busy_time = 0;
    int max_completion = 0;

    out << "\n=====" << name << " =====\n";
    out << "PID\tAT\tBT\tCT\tWT\tTAT\tRT\n";

    for(auto &pr : p) {
        total_wait += pr.waiting;
        total_turn += pr.turnaround;
        total_resp += pr.response;
        busy_time += pr.burst;
        max_completion = max(max_completion, pr.completion);

        out << pr.pid << "\t" << pr.arrival << "\t" << pr.burst << "\t"
            << pr.completion << "\t" << pr.waiting << "\t"
            << pr.turnaround << "\t" << pr.response << "\n";
    }

    double cpu_util = (double)busy_time / max_completion * 100.0;

    out << "\nAverage Waiting Time: " << total_wait/p.size() << endl;
    out << "Average Turnaround Time: " << total_turn/p.size() << endl;
    out << "Average Response Time: " << total_resp/p.size() << endl;
    out << "CPU Utilization: " << cpu_util << "%\n";
}

```

```

void calculateTimes(vector<Process> &p) {
    for(auto &pr : p) {
        pr.turnaround = pr.completion - pr.arrival;
        pr.waiting = pr.turnaround - pr.burst;
    }
}

void FCFS(vector<Process> p, ofstream &out) {
    sort(p.begin(), p.end(), [](Process a, Process b){
        return a.arrival < b.arrival;
    });

    int time = 0;
    for(auto &pr : p) {
        if(time < pr.arrival)
            time = pr.arrival;

        pr.response = time - pr.arrival;
        time += pr.burst;
        pr.completion = time;
    }

    calculateTimes(p);
    printMetrics(p, "FCFS", out);
}

void SJF(vector<Process> p, ofstream &out) {
    sort(p.begin(), p.end(), [](Process a, Process b){
        return a.burst < b.burst;
    });

    int time = 0;
    for(auto &pr : p) {
        pr.response = time;
        time += pr.burst;
        pr.completion = time;
    }

    calculateTimes(p);
    printMetrics(p, "SJF (Non Preemptive)", out);
}

```

```

}

void PriorityScheduling(vector<Process> p, ofstream &out) {
    sort(p.begin(), p.end(), [](Process a, Process b){
        return a.priority < b.priority;
    });

    int time = 0;
    for(auto &pr : p) {
        pr.response = time - pr.arrival;
        time += pr.burst;
        pr.completion = time;
    }

    calculateTimes(p);
    printMetrics(p, "Priority (Non Preemptive)", out);
}

void SRTN(vector<Process> p, ofstream &out) {
    int n = p.size();
    int completed = 0, time = 0;

    while(completed < n) {
        int idx = -1;
        int min_rem = INT_MAX;

        for(int i=0;i<n;i++) {
            if(p[i].arrival <= time && p[i].remaining > 0 &&
                p[i].remaining < min_rem) {
                min_rem = p[i].remaining;
                idx = i;
            }
        }

        if(idx == -1) {
            time++;
            continue;
        }

        if(!p[idx].started) {

```

```

        p[idx].response = time - p[idx].arrival;
        p[idx].started = true;
    }

    p[idx].remaining--;
    time++;

    if(p[idx].remaining == 0) {
        p[idx].completion = time;
        completed++;
    }
}

calculateTimes(p);
printMetrics(p, "SRTN (Preemptive SJF)", out);
}

void RR(vector<Process> p, ofstream &out, int quantum) {
    queue<int> q;
    int n = p.size();
    int time = 0, completed = 0;

    vector<bool> inQueue(n,false);

    while(completed < n) {
        for(int i=0;i<n;i++) {
            if(p[i].arrival <= time && !inQueue[i] && p[i].remaining > 0) {
                q.push(i);
                inQueue[i] = true;
            }
        }

        if(q.empty()) {
            time++;
            continue;
        }

        int idx = q.front(); q.pop();

        if(!p[idx].started) {

```

```

        p[idx].response = time - p[idx].arrival;
        p[idx].started = true;
    }

    int exec = min(quantum, p[idx].remaining);
    p[idx].remaining -= exec;
    time += exec;

    for(int i=0;i<n;i++) {
        if(p[i].arrival <= time && !inQueue[i] && p[i].remaining > 0) {
            q.push(i);
            inQueue[i] = true;
        }
    }

    if(p[idx].remaining > 0)
        q.push(idx);
    else {
        p[idx].completion = time;
        completed++;
    }
}

calculateTimes(p);
printMetrics(p, "Round Robin (Q=" + to_string(quantum) + ")", out);
}

int main() {
    cout << "Hi! Welcome to CPU Scheduling Simulator.\n";
    cout << "Scheduling algorithm – (1) FCFS (2) RR (3) SJF (4) SRTN (5) Priority (6) ALL\n";

    string choice;
    cin >> choice;

    int n;
    cout << "No. of Processes: ";
    cin >> n;

    vector<Process> processes = generateProcesses(n);

```

```

ofstream dataFile("process_StudentID_data.tex");
dataFile << "PID AT BT Priority Deadline\n";
for(auto &p : processes)
    dataFile << p.pid << " "
        << p.arrival << " "
        << p.burst << " "
        << p.priority << " "
        << p.deadline << "\n";
dataFile.close();

ofstream out("Output.tex");

if(choice=="1") FCFS(processes,out);
else if(choice=="2") RR(processes,out,2);
else if(choice=="3") SJF(processes,out);
else if(choice=="4") SRTN(processes,out);
else if(choice=="5") PriorityScheduling(processes,out);
else if(choice=="6" || choice=="ALL") {
    FCFS(processes,out);
    SJF(processes,out);
    SRTN(processes,out);
    PriorityScheduling(processes,out);
    RR(processes,out,2);
    RR(processes,out,4);
}

out.close();

cout << "DONE. Please check Output.tex\n";
}

```

```

g++ CPUSimulator.cpp -o CPUSimulator
./CPUSimulator

```



```
Feb 16 09:23
ubuntu@ubuntu:~$ nano CPUSimulator.cpp
ubuntu@ubuntu:~$ g++ CPUSimulator.cpp -o CPUSimulator
Hi! Welcome to CPU Scheduling Simulator.
Scheduling algorithm - (1) FCFS (2) RR (3) SJF (4) SRTN (5) Priority (6) ALL
1
No. of Processes: 4
DONE. Please check Output.txt
ubuntu@ubuntu:~$ cat Output.txt

===== FCFS =====
PID  AT   BT   CT   WT   TAT  RT
4     6    10   16    0   10    0
2     8     6   22    8   14    8
1     9    10   32   13   23   13
3     9     1   33   23   24   23

Average Waiting Time: 11
Average Turnaround Time: 17.75
Average Response Time: 11
CPU Utilization: 81.8182%
ubuntu@ubuntu:~$
```